1.

a. The starter code is quite messy. Some things that pop out to me initially include:
    i. There is **one** function that controls everything regarding the invoices. That function calculates amounts, calculates credits, accumulates totals, and formats text, all which could be considered different functions that should have their own independent declarations.
    ii. The comments, and lack there of, are misleading, if not just wrong! For example, line 30 says that an extra credit is added for every ten attendees, yet in the code, it is only five. The code could greatly benefit by having more comments, and cleaning up what already exists. Fowler does say that code itself does not get better from simply having more comments, but they are sorely needed here too. Probably also better naming conventions.
    iii. The chain of logic determining what genre of a play it is quite long, as well as having two different spots that it is checked. There could be a much faster, more efficient, and namely simpler way to do such a thing.

b. Running the tests was straight forward enough.

bash
pip install -r requirements.txt && python -m pytest -v

Which outputted:

tests/test_statement.py::test_example_statement PASSED [50%]
tests/test_statement.py::test_statement_with_new_play_types PASSED [100%]

========== 2 passed in 0.03s ==========

2.
Initially, the codebase had a simple two tests. After refactoring, adding characterization tests, and adding the historical feature, the total is now at nine tests, after all is done, including the new history tests.

My tests I wrote for the existing behavior:

test_tragedy_charge_and_credits_around_30_seat_threshold: checks a tragedy at 30 vs 31 seats, confirming the $10 per seat surcharge and volume credits begin only above 30 seats.

test_comedy_charge_around_20_seat_threshold: checks a comedy at 20 vs 21 seats, confirming the $100 sum and $5 per extra seat begin only above 20 seats.

Both passed against the original code and still pass after refactoring.

Tests for historical

test_spec_example_35_seats: I tested the assignment's example ($725.00, 7 credits). It also guards against accidentally adding the base over-30 credit.

test_historical_boundaries: checks 20 vs 21 seats (the $15-per-seat surcharge starts above 20) and 4 vs 5 seats (the first credit is earned at 5).

3. Refactoring
    a. **Extract Function / Replace Temp with Query** I pulled the amount calculation, volume credit calculation, and play lookup out of the loop into three named nested functions*: amount_for, volume_credits_for,* and *play_for*. I also removed the *play* and *this_amount* temporary variables and corrected a misleading comment that referred to every ten comedy attendees when the code actually divides by five. This makes the code easier to follow because each business rule has a clear name, and the loop focuses more on intent than arithmetic. All tests passed, and the approved output remained unchanged.
    b. **Split Loop / Replace Temp with Query**: I removed the total_amount and volume_credits accumulators that were updated inside the loop and replaced them with total_amount() and total_volume_credits() functions that calculate the totals across performances. This leaves the loop responsible for building the performance lines rather than formatting text and tracking totals at the same time. After running python -m pytest -q, all tests passed, and the output matched the approved file exactly.
    c. **Extract Class:** I replaced the separate amount_for and volume_credits_for functions with a PerformanceCalculator class that takes a play and audience size and provides amount and volume_credits properties. The statement() function now creates a calculator for each performance and uses it to retrieve the results. This brings the pricing and credit rules together in one place instead of scattering them across functions that each check the play type. It also lets statement() focus on coordinating the calculations and formatting the output.
    d. **Replace Conditional with Polymorphish**: I turned PerformanceCalculator into a base class with TragedyCalculator and ComedyCalculator subclasses, each handling the rules specific to its play type. A factory function selects the appropriate subclass and preserves the existing ValueError behavior for unrecognized types. This removes the if/elif chain that previously checked play types in multiple places and keeps each type's rules together. Adding

another play type can now be done by introducing a new subclass instead of modifying a long function.

3.

a. I first started by adding two new tests to test the comedy charge at the 20 seat mark, as well as testing that there is not a surcharge or credit above 30 seats. I did the math by hand, and then asserted that our code returns the expected results.

```
base) nolanpestano@Nolans-MacBook-Pro-2 python % python -m pytest -v
================================== test session starts
==================================
platform darwin -- Python 3.11.5, pytest-9.1.1, pluggy-1.7.0 --
/Users/nolanpestano/anaconda3/bin/python
cachedir: .pytest_cache
rootdir: /Users/nolanpestano/Documents/GitHub/nrpw88-refactoring/python
configfile: pytest.ini
testpaths: tests
plugins: approvaltests-0.2.4, anyio-3.5.0, dash-3.2.0, approvaltests-20.0.1
collected 4 items

tests/test_characterization.py::test_tragedy_charge_and_credits_around_30_seat_thr
eshold PASSED [ 25%]
tests/test_characterization.py::test_comedy_charge_around_20_seat_threshold
PASSED [ 50%]
tests/test_statement.py::test_example_statement PASSED                    [ 75%]
tests/test_statement.py::test_statement_with_new_play_types PASSED
[100%]

```

4.

I wrote the historical test and it all runs well with all tests passing! See the code, and my tests in pt. 2 for more details.

5.

This was a fun assignment! My reflection is on the readme.